Function (computer programming)

In computer programming, a **function** (also **procedure**, **method**, **subroutine**, **routine**, or **subprogram**) is a **callable unit**[1] of software logic that has a well-formed interface and behavior and can be invoked multiple times.

Callable units provide a powerful programming tool.[2] The primary purpose is to allow for the decomposition of a large and/or complicated problem into chunks that have relatively low cognitive load and to assign the chunks meaningful names (unless they are anonymous). Judicious application can reduce the cost of developing and maintaining software, while increasing its quality and reliability.[3]

Callable units are present at multiple levels of abstraction in the programming environment.[4] For example, a programmer may write a function in source code that is compiled to machine code that implements similar semantics. There is a callable unit in the source code and an associated one in the machine code, but they are different kinds of callable units – with different implications and features.

Terminology

Some programming languages, such as COBOL and BASIC, make a distinction between functions that return a value (typically called "functions") and those that do not (typically called "subprogram", "subroutine", or "procedure"); some, such as C, C++, and Rust, only use the term "function" irrespective of whether they return a value or not; others, such as ALGOL 60 and PL/I, only use the word *procedure*. Some object-oriented languages, such as Java and C#, refer to functions inside classes as "methods".

History

The idea of a callable unit was initially conceived by John Mauchly and Kathleen Antonelli during their work on ENIAC and recorded in a January 1947 Harvard symposium on "Preparation of Problems for EDVAC-type Machines."[5] Maurice Wilkes, David Wheeler, and Stanley Gill are generally credited with the formal invention of this concept, which they termed a *closed sub-routine*,[6][7] contrasted with an *open subroutine* or macro.[8] However, Alan Turing had discussed subroutines in a paper of 1945 on design proposals for the NPL ACE, going so far as to invent the concept of a return address stack.[9]

The idea of a subroutine was worked out after computing machines had already existed for some time. The arithmetic and conditional jump instructions were planned ahead of time and have changed relatively little, but the special instructions used for procedure calls have changed greatly over the years. The earliest computers, such as the Manchester Baby, and some early microprocessors, such as the RCA 1802, did not have a single subroutine call instruction. Subroutines could be implemented, but they required programmers to use the call sequence—a series of instructions—at each call site.

A single subroutine was implemented in 1945 in Konrad Zuse's Z4 in the form of a tape.[10]

In 1945, Alan Turing used the terms "bury" and "unbury" as a means of calling and returning from subroutines.[11][12]

In January 1947 John Mauchly presented general notes at *A Symposium of Large Scale Digital Calculating Machinery* under the joint sponsorship of Harvard University and the Bureau of Ordnance, United States Navy. Here he discusses serial and parallel operation suggesting:

> ...the structure of the machine need not be complicated one bit. It is possible, since all the logical characteristics essential to this procedure are available, to evolve a coding instruction for placing the subroutines in the memory at places known to the machine, and in such a way that they may easily be called into use.
>
> In other words, one can designate subroutine A as division and subroutine B as complex multiplication and subroutine C as the evaluation of a standard error of a sequence of numbers, and so on through the list of subroutines needed for a particular problem. ... All these subroutines will then be stored in the machine, and all one needs to do is make a brief reference to them by number, as they are indicated in the coding.[5]

Kay McNulty had worked closely with John Mauchly on the ENIAC team and developed an idea for subroutines for the ENIAC computer she was programming during World War II.[13] She and the other ENIAC programmers used the subroutines to help calculate missile trajectories.[13]

Goldstine and von Neumann wrote a paper dated 16 August 1948 discussing the use of subroutines.[14]

Some very early computers and microprocessors, such as the IBM 1620, the Intel 4004 and Intel 8008, and PIC microcontrollers, have a single-instruction subroutine call that uses a dedicated hardware stack to store return addresses; such hardware supports only a few levels of subroutine nesting, but can support recursive subroutines. Machines before the mid-1960s, such as the UNIVAC I, the PDP-1, and the IBM 1130, typically use a calling convention which saved the instruction counter in the first memory location of the called subroutine. This allows arbitrarily deep levels of subroutine nesting but does not support recursive subroutines. The IBM System/360 had a subroutine call instruction that placed the saved instruction counter value into a general-purpose register; with additional code, this can be used to support arbitrarily deep subroutine nesting and recursive subroutines. The Burroughs B5000[15] (1961) is one of the first computers to store subroutine return data on a stack.

The DEC PDP-6[16] (1964) is one of the first accumulator-based machines to have a subroutine call instruction that saved the return address in a stack addressed by an accumulator or index register. The later PDP-10 (1966), PDP-11 (1970) and VAX-11 (1976) lines followed suit; this feature also supports both arbitrarily deep subroutine nesting and recursive subroutines.[17]

Language support

In the very early assemblers, subroutine support was limited. Subroutines were not explicitly separated from each other or from the main program, and indeed the source code of a subroutine could be interspersed with that of other subprograms. Some assemblers would offer predefined macros to generate the call and return sequences. By the 1960s, assemblers usually had much more sophisticated support for both inline and separately assembled subroutines that could be linked together.

One of the first programming languages to support user-written subroutines and functions was FORTRAN II. The IBM FORTRAN II compiler was released in 1958. ALGOL 58 and other early programming languages also supported procedural programming.

Libraries

Even with this cumbersome approach, subroutines proved very useful. They allowed the use of the same code in many different programs. Memory was a very scarce resource on early computers, and subroutines allowed significant savings in the size of programs.

Many early computers loaded the program instructions into memory from a punched paper tape. Each subroutine could then be provided by a separate piece of tape, loaded or spliced before or after the main program (or "mainline"[18]); and the same subroutine tape could then be used by many different programs. A similar approach was used in computers that loaded program instructions from punched cards. The name *subroutine library* originally meant a library, in the literal sense, which kept indexed collections of tapes or decks of cards for collective use.

Return by indirect jump

To remove the need for self-modifying code, computer designers eventually provided an *indirect jump* instruction, whose operand, instead of being the return address itself, was the location of a variable or processor register containing the return address.

On those computers, instead of modifying the function's return jump, the calling program would store the return address in a variable so that when the function completed, it would execute an indirect jump that would direct execution to the location given by the predefined variable.

Jump to subroutine

Another advance was the *jump to subroutine* instruction, which combined the saving of the return address with the calling jump, thereby minimizing overhead significantly. At least five forms of this existed in the 1950s.

- Return jump: store the return address in the first word of the subroutine. Return Jump (RJ) on the UNIVAC 1103A[19] is an early example.
- Index jump: store the return address in an index register. Transfer and Set indeX (TSX) on the IBM 704[20] is an early example.
- Automatic storage into a dedicated storage location: the Store P (STP) location on the RCA 501[21] is an early example.
- Automatic storage into a designated register: The sequence and cosequence history registers (SH and CSH)[22] on the Honeywell 800 is an early example.
- Patch return address: The "return address" instruction adds one to the contents of the program counter register and patches the address portion of a branch instruction at the last word of the subroutine with the caller's return address. The "return address" instruction is followed immediately with a branch instruction to the beginning of the subroutine. The LGP-30[23] is an example.

The IBM System/360 architecture is an example of saving the return address into a designated register. The branch instructions BAL or BALR, designed for procedure calling, would save the return address in a processor register specified in the instruction, by convention register 14. To return, the subroutine had only to execute an indirect branch instruction (BR) through that register. If the subroutine needed that register for some other purpose (such as calling another subroutine), it would save the register's contents to a private memory location or a register stack.

The HP 2100 architecture is an example of saving the return address in the first word of the subroutine. The JSB instruction would save the return address in the memory location that was the target of the branch. Execution of the procedure would begin at the next memory location. In the HP 2100 assembly language, one would write, for example

```
...
JSB MYSUB    (Calls subroutine MYSUB.)
BB    ...          (Will return here after MYSUB is done.)
```

to call a subroutine called MYSUB from the main program. The subroutine would be coded as

```
MYSUB NOP          (Storage for MYSUB's return address.)
AA    ...          (Start of MYSUB's body.)
...
JMP MYSUB,I  (Returns to the calling program.)
```

The JSB instruction placed the address of the NEXT instruction (namely, BB) into the location specified as its operand (namely, MYSUB), and then branched to the NEXT location after that (namely, AA = MYSUB + 1). The subroutine could then return to the main program by executing the indirect jump JMP MYSUB, I which branched to the location stored at location MYSUB. Compilers for Fortran and other languages could easily make use of these instructions when available. This approach supported multiple levels of calls; however, since the return address, parameters, and return values of a subroutine were assigned fixed memory locations, it did not allow for recursive calls.

Incidentally, a similar method was used by Lotus 1-2-3, in the early 1980s, to discover the recalculation dependencies in a spreadsheet. Namely, a location was reserved in each cell to store the *return* address. Since circular references are not allowed for natural recalculation order, this allows a tree walk without reserving space for a stack in memory, which was very limited on small computers such as the IBM PC.

Call stack

Most modern implementations of a function call use a call stack, a special case of the stack data structure, to implement function calls and returns. Each procedure call creates a new entry, called a *stack frame*, at the top of the stack; when the procedure returns, its stack frame is deleted from the stack, and its space may be used for other procedure calls. Each stack frame contains the *private data* of the corresponding call, which typically includes the procedure's parameters and internal variables, and the return address.

The call sequence can be implemented by a sequence of ordinary instructions (an approach still used in reduced instruction set computing (RISC) and very long instruction word (VLIW) architectures), but many traditional machines designed since the late 1960s have included special instructions for that purpose.

The call stack is usually implemented as a contiguous area of memory. It is an arbitrary design choice whether the bottom of the stack is the lowest or highest address within this area, so that the stack may grow forwards or backwards in memory; however, many architectures chose the latter.

Some designs, notably some Forth implementations, used two separate stacks, one mainly for control information (like return addresses and loop counters) and the other for data. The former was, or worked like, a call stack and was only indirectly accessible to the programmer through other language constructs while the latter was more directly accessible.

When stack-based procedure calls were first introduced, an important motivation was to save precious memory.[24] With this scheme, the compiler does not have to reserve separate space in memory for the private data (parameters, return address, and local variables) of each procedure. At any moment, the stack contains only the private data of the calls that are currently *active* (namely, which have been called but haven't returned yet). Because of the ways in which programs were usually assembled from libraries, it was (and still is) not uncommon to find programs that include thousands of functions, of which only a handful are active at any given moment. For such programs, the call stack mechanism could save significant amounts of memory. Indeed, the call stack mechanism can be viewed as the earliest and simplest method for automatic memory management.

However, another advantage of the call stack method is that it allows recursive function calls, since each nested call to the same procedure gets a separate instance of its private data.

In a multi-threaded environment, there is generally more than one stack.[25] An environment that fully supports coroutines or lazy evaluation may use data structures other than stacks to store their activation records.

Delayed stacking

One disadvantage of the call stack mechanism is the increased cost of a procedure call and its matching return. The extra cost includes incrementing and decrementing the stack pointer (and, in some architectures, checking for stack overflow), and accessing the local variables and parameters by frame-relative addresses, instead of absolute addresses. The cost may be realized in increased execution time, or increased processor complexity, or both.

This overhead is most obvious and objectionable in *leaf procedures* or *leaf functions*, which return without making any procedure calls themselves.[26][27] To reduce that overhead, many modern compilers try to delay the use of a call stack until it is really needed. For example, the call of a procedure *P* may store the return address and parameters of the called procedure in certain processor registers, and transfer control to the procedure's body by a simple jump. If the procedure *P* returns without making any other call, the call stack is not used at all. If *P* needs to call another procedure *Q*, it will then use the call stack to save the contents of any registers (such as the return address) that will be needed after *Q* returns.

Features

In general, a callable unit is a list of instructions that, starting at the first instruction, executes sequentially except as directed via its internal logic. It can be invoked (called) many times during the execution of a program. Execution continues at the next instruction after the call instruction when it returns control.

Implementations

The features of implementations of callable units evolved over time and varies by context. This section describes features of the various common implementations.

General characteristics

Most modern programming languages provide features to define and call functions, including syntax for accessing such features, including:

- Delimit the implementation of a function from the rest of the program
- Assign an identifier, name, to a function
- Define formal parameters with a name and data type for each
- Assign a data type to the return value, if any
- Specify a return value in the function body
- Call a function
- Provide actual parameters that correspond to a called function's formal parameters
- Return control to the caller at the point of call
- Consume the return value in the caller
- Dispose of the values returned by a call
- Provide a private naming scope for variables
- Identify variables outside the function that are accessible within it
- Propagate an exceptional condition out of a function and to handle it in the calling context
- Package functions into a container such as module, library, object, or class

Naming

Some languages, such as Pascal, Fortran, Ada and many dialects of BASIC, use a different name for a callable unit that returns a value (*function* or *subprogram*) vs. one that does not (*subroutine* or *procedure*). Other languages, such as C, C++, C# and Lisp, use only one name for a callable unit, *function*. The C-family languages use the keyword `void` to indicate no return value.

Call syntax

If declared to return a value, a call can be embedded in an expression in order to consume the return value. For example, a square root callable unit might be called like `y = sqrt(x)`.

A callable unit that does not return a value is called as a stand-alone statement like `print("hello")`. This syntax can also be used for a callable unit that returns a value, but the return value will be ignored.

Some older languages require a keyword for calls that do not consume a return value, like `CALL print("hello")`.

Parameters

Most implementations, especially in modern languages, support parameters which the callable declares as *formal parameters*. A caller passes *actual parameters*, a.k.a. *arguments*, to match. Different programming languages provide different conventions for passing arguments.

Convention	Description	Used in
by value	A copy of the argument is passed	Default in most Algol-like languages after Algol 60, such as Pascal, Delphi, Simula, CPL, PL/M, Modula, Oberon, Ada, and many others including C, C++ and Java
by reference	A reference to the argument is passed, typically its address	Selectable in most Algol-like languages after Algol 60, such as Algol 68, Pascal, Delphi, Simula, CPL, PL/M, Modula, Oberon, Ada, and many others including C++, Fortran, PL/I
by result	The value computed during the call is copied to the argument on return	Ada OUT parameters
by value-result	A copy of the argument is passed in and the value computed during the call is copied to the argument on return	Algol, Swift in-out parameters
by name	Like a macro – replace the parameters with the unevaluated argument expressions, then evaluate the argument in the context of the caller every time that the callable uses the parameter	Algol, Scala
by constant value	Like by-value except that the parameter is treated as a constant	PL/I NONASSIGNABLE parameters, Ada IN parameters

Return value

In some languages, such as BASIC, a callable has different syntax (i.e. keyword) for a callable that returns a value vs. one that does not. In other languages, the syntax is the same regardless. In some of these languages an extra keyword is used to declare no return value, such as `void` in C, C++ and C#. In some languages, such as Python, the difference is whether the body contains a return statement with a value, and a particular callable may return with or without a value based on control flow.

Side effects

In many contexts, a callable may have side effect behavior such as modifying passed or global data, reading from or writing to a peripheral device, accessing a file, halting the program or the machine, or temporarily pausing program execution.

Side effects are considered undesirable by Robert C. Martin, who is known for promoting design principles. Martin argues that side effects can result in temporal coupling or order dependencies.[28]

In strictly functional programming languages such as Haskell, a function can have no side effects, which means it cannot change the state of the program. Functions always return the same result for the same input. Such languages typically only support functions that return a value, since there is no value in a function that has neither return value nor side effect.

Local variables

Most contexts support *local variables* – memory owned by a callable to hold intermediate values. These variables are typically stored in the call's *activation record* on the call stack along with other information such as the return address.

Nested call – recursion

If supported by the language, a callable may call itself, causing its execution to suspend while another *nested* execution of the same callable executes. Recursion is a useful means to simplify some complex algorithms and break down complex problems. Recursive languages provide a new copy of local variables on each call. If the programmer desires the recursive callable to use the same variables instead of using locals, they typically declare them in a shared context such static or global.

Languages going back to ALGOL, PL/I and C and modern languages, almost invariably use a call stack, usually supported by the instruction sets to provide an activation record for each call. That way, a nested call can modify its local variables without affecting any of the suspended call's variables.

Recursion allows direct implementation of functionality defined by mathematical induction and recursive divide and conquer algorithms. Here is an example of a recursive function in C to find Fibonacci numbers:

```
int fibonacci(unsigned int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

Early languages like Fortran did not initially support recursion because only one set of variables and return address were allocated for each callable.[29] Early computer instruction sets made storing return addresses and variables on a stack difficult. Machines with index registers or general-purpose registers, e.g., CDC 6000 series, PDP-6, GE 635, System/360, UNIVAC 1100 series, could use one of those registers as a stack pointer.

Nested scope

Some languages, e.g., Ada, Pascal, PL/I, Python, support declaring and defining a function inside, e.g., a function body, such that the name of the inner is only visible within the body of the outer.

A simple example in Pascal:

```
function E(x: real): real;
    function F(y: real): real;
    begin
        F := x + y
    end;
begin
    E := F(3) + F(4)
end;
```

The function F is nested within E. Note that E's parameter x is also visible in F (as F is a part of E) while both x and y are invisible outside E and F respectively.

Reentrancy

If a callable can be executed properly even when another execution of the same callable is already in progress, that callable is said to be *reentrant*. A reentrant callable is also useful in multi-threaded situations since multiple threads can call the same callable without fear of interfering with each other. In the IBM CICS transaction processing system, *quasi-reentrant* was a slightly less restrictive, but similar, requirement for application programs that were shared by many threads.

Overloading

Some languages support *overloading* – allow multiple callables with the same name in the same scope, but operating on different types of input. Consider the square root function applied to real number, complex number and matrix input. The algorithm for each type of input is different, and the return value may have a different type. By writing three separate callables with the same name. i.e. *sqrt*, the resulting code may be easier to write and to maintain since each one has a name that is relatively easy to understand and to remember instead of giving longer and more complicated names like *sqrt_real*, *sqrt_complex*, *qrt_matrix*.

Overloading is supported in many languages that support strong typing. Often the compiler selects the overload to call based on the type of the input arguments or it fails if the input arguments do not select an overload. Older and weakly-typed languages generally do not support overloading.

Here is an example of overloading in C++, two functions `area` that accept different types:

```
// returns the area of a rectangle defined by height and width
double area(double height, double width) {
    return h * w;
}

// returns the area of a circle defined by radius
double area(double radius) {
    return radius * radius * std::numbers::pi;
}

int main() {
    double rectangleArea = area(3, 4);
    double circleArea = area(5);
}
```

PL/I has the `GENERIC` attribute to define a generic name for a set of entry references called with different types of arguments. Example:

```
DECLARE gen_name GENERIC(
    name  WHEN(FIXED BINARY),
    flame  WHEN(FLOAT),
    pathname OTHERWISE);
```

Multiple argument definitions may be specified for each entry. A call to "gen_name" will result in a call to "name" when the argument is FIXED BINARY, "flame" when FLOAT", etc. If the argument matches none of the choices "pathname" will be called.

Closure

A *closure* is a callable plus values of some of its variables captured from the environment in which it was created. Closures were a notable feature of the Lisp programming language, introduced by John McCarthy. Depending on the implementation, closures can serve as a mechanism for side-effects.

Exception reporting

Besides its happy path behavior, a callable may need to inform the caller about an exceptional condition that occurred during its execution.

Most modern languages support exceptions which allows for exceptional control flow that pops the call stack until an exception handler is found to handle the condition.

Languages that do not support exceptions can use the return value to indicate success or failure of a call. Another approach is to use a well-known location like a global variable for success indication. A callable writes the value and the caller reads it after a call.

In the IBM System/360, where return code was expected from a subroutine, the return value was often designed to be a multiple of 4—so that it could be used as a direct branch table index into a branch table often located immediately after the call instruction to avoid extra conditional tests, further improving efficiency. In the System/360 assembly language, one would write, for example:

```
            BAL  14, SUBRTN01    go to a subroutine, storing return address in R14
            B    TABLE(15)      use returned value in reg 15 to index the branch table,
*                               branching to the appropriate branch instr.
TABLE       B    OK             return code =00   GOOD                  }
            B    BAD            return code =04   Invalid input         } Branch table
            B    ERROR          return code =08   Unexpected condition  }
```

Call overhead

A call has runtime overhead, which may include but is not limited to:

- Allocating and reclaiming call stack storage
- Saving and restoring processor registers
- Copying input variables
- Copying values after the call into the caller's context
- Automatic testing of the return code
- Handling of exceptions
- Dispatching such as for a virtual method in an object-oriented language

Various techniques are employed to minimize the runtime cost of calls.

Compiler optimization

Some optimizations for minimizing call overhead may seem straight forward, but cannot be used if the callable has side effects. For example, in the expression `(f(x)-1)/(f(x)+1)`, the function `f` cannot be called only once with its value used two times since the two calls may return different results. Moreover, in the few languages which define the order of evaluation of the division operator's operands, the value of `x` must be fetched again before the second call, since the first call may have changed it.

Determining whether a callable has a side effect is difficult – indeed, undecidable by virtue of Rice's theorem. So, while this optimization is safe in a purely functional programming language, a compiler for a language not limited to functional typically assumes the worst case, that every callable may have side effects.

Inlining

Inlining eliminates calls for particular callables. The compiler replaces each call with the compiled code of the callable. Not only does this avoid the call overhead, but it also allows the compiler to optimize code of the caller more effectively by taking into account the context and arguments at that call. Inlining, however, usually increases the compiled code size, except when only called once or the body is very short, like one line.

Sharing

Callables can be defined within a program, or separately in a library that can be used by multiple programs.

Inter-operability

A compiler translates call and return statements into machine instructions according to a well-defined calling convention. For code compiled by the same or a compatible compiler, functions can be compiled separately from the programs that call them. The instruction sequences corresponding to call and return statements are called the procedure's prologue and epilogue.

Built-in functions

A *built-in function*, or *builtin function*, or *intrinsic function*, is a function for which the compiler generates code at compile time or provides in a way other than for other functions.[30] A built-in function does not need to be defined like other functions since it is *built in* to the programming language.[31]

compile time evaluated functions

In some programming languages, for instance C++, Rust and Zig, functions can be evaluated at compile time; reducing the runtime overhead in the ways such as not using additional stack frame, using less memory and baking the return values to the binary code itself.[32]

Programming

Trade-offs

Advantages

Advantages of breaking a program into functions include:

- Decomposing a complex programming task into simpler steps: this is one of the two main tools of structured programming, along with data structures
- Reducing duplicate code within a program
- Enabling reuse of code across multiple programs
- Dividing a large programming task among various programmers or various stages of a project
- Hiding implementation details from users of the function
- Improving readability of code by replacing a block of code with a function call where a descriptive (https://books.google.com/books?id=_i6bDeoCQzsC&pg=PA39&dq=descriptive+function+name) function name serves to describe the block of code. This makes the calling code concise and readable even if the function is not meant to be reused.
- Improving traceability (i.e. most languages offer ways to obtain the call trace which includes the names of the involved functions and perhaps even more information such as file names and line numbers); by not decomposing the code into functions, debugging would be severely impaired

Disadvantages

Compared to using in-line code, invoking a function imposes some computational overhead in the call mechanism.

A function typically requires standard housekeeping code – both at the entry to, and exit from, the function (function prologue and epilogue – usually saving general purpose registers and return address as a minimum).

Conventions

Many programming conventions have been developed regarding callables.

With respect to naming, many developers name a callable with a phrase starting with a verb when it does a certain task, with an adjective when it makes an inquiry, and with a noun when it is used to substitute variables.

Some programmers suggest that a callable should perform exactly one task, and if it performs more than one task, it should be split up into multiple callables. They argue that callables are key components in software maintenance, and their roles in the program must remain distinct.

Proponents of modular programming advocate that each callable should have minimal dependency on the rest of the codebase. For example, the use of global variables is generally deemed unwise, because it adds coupling between all callables that use the global variables. If such coupling is not necessary, they advise to refactor callables to accept passed parameters instead.

Examples

Early BASIC

Early BASIC variants require each line to have a unique number (line number) that orders the lines for execution, provides no separation of the code that is callable, no mechanism for passing arguments or to return a value and all variables are global. It provides the command `GOSUB` where *sub* is short for *sub procedure*, *subprocedure* or *subroutine*. Control jumps to the specified line number and then continues on the next line on return.

```
10 REM A BASIC PROGRAM
20 GOSUB 100
30 GOTO 20
100 INPUT “GIVE ME A NUMBER”; N
110 PRINT “THE SQUARE ROOT OF”; N;
120 PRINT “IS”; SQRT(N)
130 RETURN
```

This code repeatedly asks the user to enter a number and reports the square root of the value. Lines 100-130 are the callable.

Small Basic

In Microsoft Small Basic, targeted to the student first learning how to program in a text-based language, a callable unit is called a *subroutine*. The `Sub` keyword denotes the start of a subroutine and is followed by a name identifier. Subsequent lines are the body which ends with the `EndSub` keyword.[33]

```
Sub SayHello
    TextWindow.WriteLine("Hello!")
EndSub
```

This can be called as `SayHello()`. [34]

Visual Basic

In later versions of Visual Basic (VB), including the latest product line and VB6, the term *procedure* is used for the callable unit concept. The keyword `Sub` is used to return no value and `Function` to return a value. When used in the context of a class, a procedure is a method. [35]

Each parameter has a data type that can be specified, but if not, defaults to `Object` for later versions based on .NET and variant for VB6.[36]

VB supports parameter passing conventions by value and by reference via the keywords `ByVal` and `ByRef`, respectively. Unless `ByRef` is specified, an argument is passed `ByVal`. Therefore, `ByVal` is rarely explicitly specified.

For a simple type like a number these conventions are relatively clear. Passing `ByRef` allows the procedure to modify the passed variable whereas passing `ByVal` does not. For an object, semantics can confuse programmers since an object is always treated as a reference. Passing an object `ByVal` copies the reference, not the state of the object. The called procedure can modify the state of the object via its methods yet cannot modify the object reference of the actual parameter.

```
Sub DoSomething()
    ' Some Code Here
End Sub
```

The does not return a value and has to be called stand-alone, like `DoSomething`

```
Function GiveMeFive() as Integer
    GiveMeFive= 5
End Function
```

This returns the value 5, and a call can be part of an expression like `y = x + GiveMeFive()`

```
Sub AddTwo(ByRef intValue as Integer)
    intValue = intValue + 2
End Sub
```

This has a side-effect – modifies the variable passed by reference and could be called for variable `v` like `AddTwo(v)`. Giving v is 5 before the call, it will be 7 after.

C and C++

In C and C++, a callable unit is called a *function*. A function definition starts with the name of the type of value that it returns or `void` to indicate that it does not return a value. This is followed by the function name, formal arguments in parentheses, and body lines in braces.

In C++, a function declared in a class (as non-static) is called a *member function* or *method*. A function outside of a class can be called a *free function* to distinguish it from a member function.[37]

```
void doSomething() {
    // some code
}
```

This function does not return a value and is always called stand-alone, like `doSomething()`

```
int returnFive() {
    return 5;
}
```

This function returns the integer value 5. The call can be stand-alone or in an expression like `y = x + returnFive()`

```
void addTwo(int* pi) {
    *pi += 2;
}
```

This function has a side-effect – modifies the value passed by address to the input value plus 2. It could be called for variable `v` as `addTwo(&v)` where the ampersand (&) tells the compiler to pass the address of a variable. Giving v is 5 before the call, it will be 7 after.

```
void addTwo(int& i) {
    i += 2;
}
```

This function requires C++ – would not compile as C. It has the same behavior as the preceding example but passes the actual parameter by reference rather than passing its address. A call such as `addTwo(v)` does not include an ampersand since the compiler handles passing by reference without syntax in the call.

PL/I

In PL/I a called procedure may be passed a *descriptor* providing information about the argument, such as string lengths and array bounds. This allows the procedure to be more general and eliminates the need for the programmer to pass such information. By default PL/I passes arguments by reference. A (trivial) function to change the sign of each element of a two-dimensional array might look like:

```
change_sign: procedure(array);
  declare array(*,*) float;
  array = -array;
end change_sign;
```

This could be called with various arrays as follows:

```
/* first array bounds from -5 to +10 and 3 to 9 */
declare array1 (-5:10, 3:9)float;
/* second array bounds from 1 to 16 and 1 to 16 */
declare array2 (16,16) float;
call change_sign(array1);
call change_sign(array2);
```

Python

In Python, the keyword `def` denotes the start of a function definition. The statements of the function body follow as indented on subsequent lines and end at the line that is indented the same as the first line or end of file.[38]

```
def format_greeting(name: str) -> None:
    return f"Welcome, {name}!"

def greet_martin() -> None:
    print(format_greeting("Martin"))
```

The first function returns greeting text that includes the name passed by the caller. The second function calls the first and is called like `greet_martin()` to write "Welcome Martin" to the console.

Prolog

In the procedural interpretation of logic programs, logical implications behave as goal-reduction procedures. A rule (or clause) of the form:

```
A :- B
```

which has the logical reading:

```
A if B
```

behaves as a procedure that reduces goals that unify with `A` to subgoals that are instances of`B`.

Consider, for example, the Prolog program:

```
mother_child(elizabeth, charles).
father_child(charles, william).
father_child(charles, harry).
parent_child(X, Y) :- mother_child(X, Y).
parent_child(X, Y) :- father_child(X, Y).
```

Notice that the motherhood function, `X = mother(Y)` is represented by a relation, as in a relational database. However, *relations* in Prolog *function* as callable units.

For example, the procedure call `?- parent_child(X, charles)` produces the output `X = elizabeth`. But the same procedure can be called with other input-output patterns. For example:

```
?- parent_child(elizabeth, Y).
Y = charles.

?- parent_child(X, Y).
X = elizabeth,
Y = charles.

X = charles,
Y = harry.

X = charles,
Y = william.

?- parent_child(william, harry).
no.

?- parent_child(elizabeth, charles).
yes.
```

See also

- Asynchronous procedure call – function that is called after its parameters are set by other activities
- Command–query separation – IT architecture separating actions and reads (CQS)
- Compound operation – Basic programming language construct
- Coroutine – Functions whose execution you can pause
- Evaluation strategy – Programming language evaluation rules
- Event (computing) – Computing state associated with a point in time

- Function (mathematics) – Association of one output to each input
- Functional programming – Programming paradigm based on applying and composing functions
- Fused operation – Basic programming language construct
- Generator (computer programming) – Routine that generates a sequence of values
- Intrinsic function – Function whose implementation is handled specially by the compiler
- Lambda function (computer programming) – Function definition that is not bound to an identifier
- Logic programming – Programming paradigm based on formal logic
- Modular programming – Organizing code into modules
- Operator overloading – Feature of some programming languages
- Parameter (computer programming)
- Protected procedure
- Transclusion – Including one data set inside another automatically

References

1. "Terminology Glossary" (https://pages.nist.gov/ElectionGlossary/). *nist.gov*. NIST. Retrieved 9 February 2024. "Callable unit: (Of a software program or logical design) Function, method, operation, subroutine, procedure, or analogous structural unit that appears within a module."
2. Donald E. Knuth (1997). *The Art of Computer Programming, Volume I: Fundamental Algorithms*. Addison-Wesley. ISBN 0-201-89683-4.
3. O.-J. Dahl; E. W. Dijkstra; C. A. R. Hoare (1972). *Structured Programming*. Academic Press. ISBN 0-12-200550-3.
4. "What is Abstraction in Programming? Explained for Beginners" (https://www.freecodecamp.org/news/what-is-abstraction-in-programming-for-beginners/). *freeCodeCamp.org*. 21 December 2022. Retrieved 25 November 2025.
5. Mauchly, J.W. (1982). "Preparation of Problems for EDVAC-Type Machines" (https://archive.org/details/originsofdigital0000rand/page/365). In Randell, Brian (ed.). *The Origins of Digital Computers*. Springer. pp. 393–397. doi:10.1007/978-3-642-61812-3_31 (https://doi.org/10.1007%2F978-3-642-61812-3_31). ISBN 978-3-642-61814-7.
6. Wheeler, D. J. (1952). "The use of sub-routines in programmes" (http://www.laputan.org/pub/papers/wheeler.pdf) (PDF). *Proceedings of the 1952 ACM national meeting (Pittsburgh) on - ACM '52*. p. 235. doi:10.1145/609784.609816 (https://doi.org/10.1145%2F609784.609816).
7. Wilkes, M. V.; Wheeler, D. J.; Gill, S. (1951). *Preparation of Programs for an Electronic Digital Computer*. Addison-Wesley.
8. Dainith, John (2004). " "open subroutine." A Dictionary of Computing" (http://www.encyclopedia.com/doc/1O11-opensubroutine.html). *Encyclopedia.com*. Retrieved 14 January 2013.
9. Turing, Alan M. (1945), *Report by Dr. A.M. Turing on proposals for the development of an Automatic Computing Engine (ACE): Submitted to the Executive Committee of the NPL in February 1946* reprinted in Copeland, B. J., ed. (2005). *Alan Turing's Automatic Computing Engine* (https://archive.org/details/alanturingsautom0000unse). Oxford: Oxford University Press. p. 383. ISBN 0-19-856593-3.
10. Speiser, Ambros Paul (2002). "Konrad Zuse's Z4: Architecture, Programming and Modifications at ETH Zurich". In Rojas, Rául; Hashagen, Ulf [in German] (eds.). *The first Computers: History and Architectures*. MIT. pp. 263–276. ISBN 978-0-262-18197-6.

11. Turing, Alan Mathison (19 March 1946) [1945], *Proposals for Development in the Mathematics Division of an Automatic Computing Engine (ACE)* (NB. Presented on 1946-03-19 before the Executive Committee of the National Physical Laboratory (Great Britain).)
12. Carpenter, Brian Edward; Doran, Robert William (1 January 1977) [October 1975]. "The other Turing machine" (https://doi.org/10.1093%2Fcomjnl%2F20.3.269). *The Computer Journal*. **20** (3): 269–279. doi:10.1093/comjnl/20.3.269 (https://doi.org/10.1093%2Fcomjnl%2F20.3.269). (11 pages)
13. Isaacson, Walter (18 September 2014). "Walter Isaacson on the Women of ENIAC" (https://web.archive.org/web/20181212003245/http://fortune.com/2014/09/18/walter-isaacson-the-women-of-eniac/). *Fortune*. Archived from the original (http://fortune.com/2014/09/18/walter-isaacson-the-women-of-eniac/) on 12 December 2018. Retrieved 14 December 2018.
14. Herman H. Goldstine; John von Neumann (1947). "Volume 3, 12.0 COMBINING ROUTINES" (https://www.ias.edu/sites/default/files/library/pdfs/ecp/planningcodingof0103inst.pdf#page=163) (PDF). *Report on the Mathematical and Logical aspects of an Electronic Computing Instrument - Part II, Volume I-3, Planning and Coding of Problems for an Electronic Computing Instrument* (https://www.ias.edu/sites/default/files/library/pdfs/ecp/planningcodingof0103inst.pdf) (PDF) (Technical report). Institute for Advanced Study.
15. *The Operational Characteristics of the Processors for the Burroughs B5000* (http://bitsavers.org/pdf/burroughs/LargeSystems/B5000_5500_5700/5000-21005_B5000_operChar_1963.pdf) (PDF). Revision A. Burroughs Corporation. 1963. 5000-21005. Retrieved 8 February 2024.
16. "Push-Down Instructions" (http://bitsavers.org/pdf/dec/pdp6/F-65_PDP-6_Handbook_Aug64.pdf#page=35) (PDF). *Programmed Data Processor 6 - Handbook* (http://bitsavers.org/pdf/dec/pdp6/F-65_PDP-6_Handbook_Aug64.pdf) (PDF). p. 37. Retrieved 8 February 2024.
17. Guy Lewis Steele Jr. AI Memo 443. 'Debunking the "Expensive Procedure Call" Myth; or, Procedure call implementations considered harmful" (http://dspace.mit.edu/bitstream/handle/1721.1/5753/AIM-443.pdf?sequence=2). Section "C. Why Procedure Calls Have a Bad Reputation".
18. Frank, Thomas S. (1983). *Introduction to the PDP-11 and Its Assembly Language* (https://books.google.com/books?id=YN4mAAAAMAAJ). Prentice-Hall software series. Prentice-Hall. p. 195. ISBN 9780134917047. Retrieved 6 July 2016. "We could supply our assembling clerk with copies of the source code for all of our useful subroutines and then when presenting him with a mainline program for assembly, tell him which subroutines will be called in the mainline [...]"
19. *The UNIVAC Scientific General Purpose Computer System (Model 1103 A) - Preliminary Information* (http://bitsavers.org/pdf/univac/1103/1103A_prelimInfo_Dec55.pdf) (PDF). Remington Rand Engineering Research Associates Division. 1 December 1955. p. 1-10 (http://bitsavers.org/pdf/univac/1103/1103A_prelimInfo_Dec55.pdf#12). Retrieved 8 October 2025.
20. "Control Operations" (http://bitsavers.org/pdf/ibm/704/24-6661-2_704_Manual_1955.pdf#page=23) (PDF). *704 electronic data-processing machine - manual of operation* (http://bitsavers.org/pdf/ibm/704/24-6661-2_704_Manual_1955.pdf) (PDF). IBM. 1955. p. 25 (http://bitsavers.org/pdf/ibm/704/24-6661-2_704_Manual_1955.pdf#page=25). 24-6661-2. Retrieved 8 October 2025.
21. "Automatic Storage of Contents of P Register (STP)" (http://bitsavers.org/pdf/rca/501/P501-2_RCA501_ProgrammerRef_Nov58.pdf#page=24) (PDF). *RCA 501 Electronic Data processing System - Programmers' Reference Manual* (http://bitsavers.org/pdf/rca/501/P501-2_RCA501_ProgrammerRef_Nov58.pdf) (PDF). RCA Electronic Data Processing Division. 1958. p. 18. P501-2. Retrieved 8 October 2025.
22. "History Registers" (http://bitsavers.org/pdf/honeywell/h800/H800_programmersRefMan.pdf#page=63) (PDF). *Honeywell 800 Transistorized Data processing System - Programmers' Reference Manual* (http://bitsavers.org/pdf/honeywell/h800/H800_programmersRefMan.pdf) (PDF). Honeywell. 1960. p. 55. DSI-31. Retrieved 8 October 2025.

23. *Royal Precision Electronic Computer LGP - 30 Programming Manual* (https://ed-thelen.org/comp-hist/lgp-30-man.html). Port Chester, New York: Royal Precision. April 1957. Retrieved 19 November 2023.
24. Tanenbaum, Andrew S.; Austin, Todd (2013). *Structured computer organization*. Always learning (6th ed.). Boston, Mass. Munich: Pearson. ISBN 978-0-13-291652-3.
25. Buttlar, Dick; Farrell, Jacqueline; Nichols, Bradford (1996). *PThreads Programming: A POSIX Standard for Better Multiprocessing* (https://books.google.com/books?id=vjgEEAAAQBAJ&pg=PA2). O'Reilly Media, Inc. pp. 2–5. ISBN 978-1-4493-6475-5. OCLC 1036778036 (https://search.worldcat.org/oclc/1036778036).
26. "ARM Information Center" (http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.faqs/ka13785.html). Infocenter.arm.com. Retrieved 29 September 2013.
27. "x64 stack usage" (https://learn.microsoft.com/en-us/cpp/build/stack-usage?view=msvc-170#function-types). *Microsoft Learn*. Microsoft. Function types. Retrieved 29 October 2025.
28. Martin, Robert C. (1 August 2008). *Clean Code: A Handbook of Agile Software Craftsmanship* (https://www.oreilly.com/library/view/clean-code-a/9780136083238/) (1 ed.). Pearson. ISBN 9780132350884. Retrieved 19 May 2024.
29. Verhoeff, Tom (2018). "A Master Class on Recursion" (https://books.google.com/books?id=Z_5sDwAAQBAJ&pg=PA616). In Böckenhauer, Hans-Joachim; Komm, Dennis; Unger, Walter (eds.). *Adventures Between Lower Bounds and Higher Altitudes: Essays Dedicated to Juraj Hromkovič on the Occasion of His 60th Birthday*. Springer. p. 616. ISBN 978-3-319-98355-4. OCLC 1050567095 (https://search.worldcat.org/oclc/1050567095).
30. "Built-in functions" (https://www.ibm.com/docs/en/zos/2.2.0?topic=lf-built-in-functions). *ibm.com*. 9 March 2017. Retrieved 25 December 2023.
31. *Study Material Python* (https://books.google.com/books?id=d0nhEAAAQBAJ&pg=PA87). April 2023. p. 87. Retrieved 25 December 2023.
32. Microsoft Learn (22 February 2023). "constexpr (C++)" (https://learn.microsoft.com/en-us/cpp/cpp/constexpr-cpp). *learn.microsoft.com*. Microsoft Learn.
33. "Small Basic" (https://smallbasic-publicwebsite.azurewebsites.net/). *Small Basic*. Retrieved 8 February 2024.
34. "Small Basic Getting Started Guide: Chapter 9: Subroutines" (https://social.technet.microsoft.com/wiki/contents/articles/16077.small-basic-getting-started-guide-chapter-9-subroutines.aspx). Microsoft. 17 January 2024.
35. "Procedures in Visual Basic" (https://learn.microsoft.com/en-us/dotnet/visual-basic/programming-guide/language-features/procedures/). *Microsoft Learn*. 15 September 2021. Retrieved 8 February 2024.
36. "Dim statement (Visual Basic)" (https://learn.microsoft.com/en-us/dotnet/visual-basic/language-reference/statements/dim-statement). *Microsoft Learn*. 15 September 2021. Retrieved 8 February 2024.
37. "what is meant by a free function" (https://stackoverflow.com/questions/4861914/what-is-the-meaning-of-the-term-free-function-in-c).
38. "4. More Control Flow Tools — Python 3.9.7 documentation" (https://docs.python.org/3.9/tutorial/controlflow.html#defining-functions).

Retrieved from "https://en.wikipedia.org/w/index.php?title=Function_(computer_programming)&oldid=1352023430"